A

AetherDB V1.31.2

Worldview Engine / 世界观数据库与创作引擎

项目概况说明与介绍

Project Profile Document

AMD Hackathon Submission / AMD 黑客马拉松项目资料

AetherDB Version 1.31.2 | Generated 2026-08-02

双语格式说明

每个中文说明均位于对应英文说明上方，便于多国评委快速审阅。

Each Chinese statement is followed by its English translation for international review.

执行摘要
Executive Summary

AetherDB V1.31.2 是面向长篇小说、游戏叙事和 IP 世界观管理的本地优先创作系统。它把写作工作台、世界观数据库、语义检索、多模态素材管理和 AI 辅助流程整合到一个桌面应用中。

AetherDB V1.31.2 is a local-first authoring system for long-form fiction, game narrative, and IP worldbuilding. It combines a writing workspace, structured lore database, semantic retrieval, multimodal asset management, and AI-assisted workflows in one desktop application.

当前文档以 V1.31.2 项目的实际模块化实现为依据。历史技术路线图只作为参考，不作为当前功能状态的判定来源。

This document is based on the actual modular implementation of V1.31.2. Historical roadmaps are treated as references only and are not used as the source of truth for current feature status.

后端采用 FastAPI，静态统计约 279 个 `/api/*` method/path；前端采用 Vue 3 + Tauri 2，通过本地回环服务与后端通信。

The backend uses FastAPI with about 279 statically counted `/api/*` method/path entries; the frontend uses Vue 3 + Tauri 2 and communicates with the backend through a local loopback service.

中文 / Chinese	English
创作、检索、设定管理和 AI 辅助在一个桌面工作流中完成。	Writing, retrieval, lore management, and AI assistance live in a single desktop workflow.
前后端分离，后端用严格认证和能力权限控制关键操作。	Frontend and backend are separated; backend operations are controlled by strict authentication and capability gates.
本地数据库和本地模型优先，云端模型仅作为可配置供应商。	Local data and local models are prioritized; cloud models are configurable providers rather than hard dependencies.
ROCm 适配以可验证执行证据为准，不以 UI 状态或配置声明为准。	ROCm support is based on verifiable execution evidence, not UI labels or configuration claims.

1. 项目背景

1. Project Background

长篇创作项目通常会经历角色数量膨胀、设定跨卷变化、关系网络复杂化和时间线冲突。传统文档或表格难以同时承担写作入口、知识库、检索系统和一致性检查工具。

Long-form creative projects often suffer from expanding character sets, evolving lore across volumes, complex relationship networks, and timeline conflicts. Traditional documents or spreadsheets cannot reliably serve as a writing interface, knowledge base, retrieval system, and consistency checker at once.

AetherDB 的目标是把“世界观圣经”从静态资料夹升级为可检索、可推理、可验证、可打包交付的本地应用，使作者能在同一个环境里完成创作、设定沉淀和 AI 辅助校验。

AetherDB upgrades the world bible from a static folder into a searchable, reasoned, verifiable, and distributable local application where authors can write, preserve lore, and use AI-assisted checks in the same environment.

核心问题

Core Problems

• 设定散落在正文、笔记、聊天记录和素材文件中，后期很难追踪。

Lore is scattered across chapters, notes, chats, and asset files, making later tracking difficult.

• AI 可以加速整理，但如果没有审核流程，容易污染正式设定。

AI can accelerate organization, but without review gates it can contaminate canonical lore.

• 创作者需要低门槛桌面工具，而不是要求每位作者维护复杂服务端部署。

Writers need a low-friction desktop tool, not a complex server deployment they must maintain.

• 比赛环境需要明确证明 AMD Radeon GPU/ROCm 路径真实可用，不能只展示配置项。

The competition environment needs clear proof that the AMD Radeon GPU/ROCm path is actually usable, not merely configured.

设计原则

Design Principles

• 本地优先：作品数据库、模型包和运行状态默认保留在本机。

Local-first: project databases, model packages, and runtime state stay on the user machine by default.

• 作者主权：AI 只提供分析、抽取、建议和预览，关键写入需要确认。

Author sovereignty: AI provides analysis, extraction, suggestions, and previews; critical writes require confirmation.

• 实际模块优先：PDF 以当前源码、接口、模型包和验证脚本为依据，而不是依赖过期路线图。

Implementation-first: this PDF uses current source modules, APIs, model manifests, and verification scripts rather than outdated roadmap claims.

• 安全默认：API Key 不进 URL，本地密钥生成和桌面 owner token 分离。

Secure defaults: API keys never travel in URLs; generated local keys and desktop owner tokens are separated.

2. 目标用户和应用场景

2. Target Users and Use Cases

用户类型	痛点与使用方式	User / Usage
职业小说作者	维护多卷角色、地点、组织、事件、规则和伏笔；在写作时快速检索设定。	Professional novelists manage multi-volume characters, locations, organizations, events, rules, and foreshadowing while writing.
网文/系列化团队	多人协作时需要统一世界观数据库、变更记录和可审查导入流程。	Serialized fiction teams need a shared lore database, change history, and reviewable import flow.
游戏叙事与 IP 设定团队	管理阵营、道具、地理、任务线、时间线和美术素材检索。	Game narrative and IP teams manage factions, props, geography, quest lines, timelines, and visual asset retrieval.
编辑与资料整理人员	把已有文本批量导入，抽取实体和关系，建立可检索资料库。	Editors and lore managers import existing text, extract entities and relations, and build searchable references.

典型应用场景

Representative Scenarios

- 创作工作台：作者先写正文，随后触发 AI 分析、实体抽取和冲突检查。

Writing workspace: the author writes first, then triggers AI analysis, entity extraction, and conflict checks.

- 世界观管理：角色、地点、势力、事件、物品、概念和规则以结构化实体沉淀。

Worldbuilding management: characters, places, factions, events, items, concepts, and rules become structured entities.

- 跨章节检索：BGE-M3 向量和 FTS5 关键词检索共同支持 RAG 上下文召回。

Cross-chapter retrieval: BGE-M3 vectors and FTS5 keyword search support RAG context retrieval.

- 多模态素材：BGE-VL 支持文本/图像检索通道，服务角色图、地图和参考图管理。

Multimodal assets: BGE-VL supports text/image retrieval channels for character art, maps, and references.

- 音频预览：Kokoro TTS 作为轻量基线，用于章节朗读、语音预览和发音词典验证。

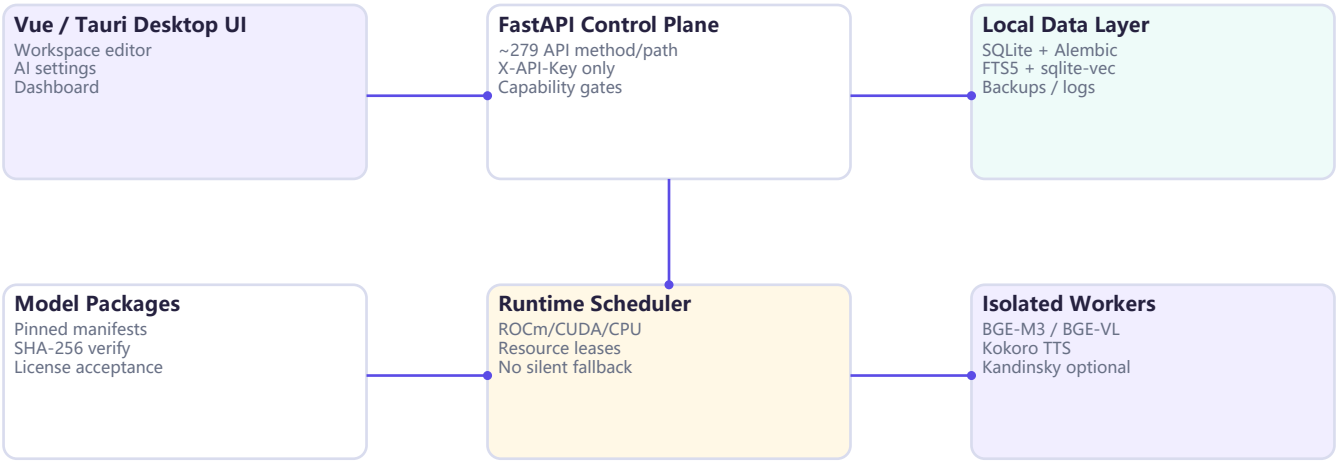
Audio preview: Kokoro TTS is a lightweight baseline for chapter narration, voice preview, and pronunciation dictionary checks.

3. 系统架构

3. System Architecture

系统采用桌面前端、后端控制面、数据层、模型包和隔离 Worker 的分层结构。前端负责创作体验和状态呈现，后端负责权限、数据一致性、模型调度和安全边界。

The system is layered into a desktop frontend, backend control plane, data layer, model packages, and isolated workers. The frontend owns authoring experience and status display; the backend owns permissions, consistency, model scheduling, and security boundaries.



关键模块

Key Modules

• API 控制面：FastAPI 当前统计约 279 个 `/api/*` method/path，覆盖创作、导入、搜索、模型管理、媒体、音频和维护。

API control plane: FastAPI currently exposes about 279 `/api/*` method/path entries covering writing, import, search, model management, media, audio, and maintenance.

• 数据层：SQLite 承担本地数据库，Alembic 管理迁移；FTS5 与 sqlite-vec 支持关键词和向量检索。

Data layer: SQLite stores local project data, Alembic manages migrations, and FTS5 plus sqlite-vec enable keyword and vector retrieval.

• 模型层：模型包由 manifest 描述固定版本、许可、后端、大小和 SHA-256，安装后才能进入调度。

Model layer: model packages declare pinned version, license, backend, size, and SHA-256 in manifests, and must be verified before scheduling.

• 桌面层：Tauri 启动本地后端，使用动态回环端口、握手和 owner token 控制桌面会话。

Desktop layer: Tauri launches the local backend with a dynamic loopback port, handshake, and owner token for the desktop session.

安全与权限

Security and Permissions

• 认证只接受 `X-API-Key` 请求头，禁用 URL 查询参数中的 key，降低浏览器历史和日志泄露风险。

Authentication accepts only the `X-API-Key` header and rejects query-string keys to reduce browser history and log leakage.

• 本地开发或桌面运行会生成本机密钥文件；发行包不会包含私有 API key 或本地配置。

Local development or desktop runtime can generate local key files; release packages do not contain private API keys or local configs.

• 管理员能力、模型执行能力和系统维护能力通过 capability gate 控制，而不是只依赖前端按钮状态。

Admin, model execution, and maintenance operations are guarded by backend capability gates, not merely frontend button state.

• 前端与后端分离，因此非开发环境 UI 问题不会改变后端认证、数据写入和模型调度的安全边界。

Frontend/backend separation means non-development UI issues do not change backend authentication, data writes, or model scheduling boundaries.

4. 模型及算法介绍

4. Models and Algorithms

AetherDB 把模型视为可验证资源，而不是散落在环境中的隐式依赖。模型能力通过 manifest、registry、runtime scheduler 和 isolated worker 串联。

AetherDB treats models as verifiable resources rather than implicit environment dependencies. Model capabilities are connected through manifests, registry, runtime scheduler, and isolated workers.

模型包	用途	后端 / Backends	说明 / Notes
AetherDB BGE-M3 Multilingual Embedding	文本语义向量、RAG 上下文、混合搜索。	rocm_migraphx, cuda, cpu	Text embeddings, RAG context, and hybrid search. Adapter: aetherdb_onnx_embedding Size: 2.13 GiB
AetherDB BGE-VL Base	图像/文本多模态检索，角色图、地图和素材管理。	rocm_pytorch, cuda, cpu	Image/text multimodal retrieval for character art, maps, and assets. Adapter: aetherdb_torch_bge_vl Size: 0.28 GiB
AetherDB Kandinsky 2.2	可选图像生成能力，面向高显存 ROCm/CUDA 环境。	rocm_pytorch, cuda	Optional image generation for high-VRAM ROCm/CUDA environments. Adapter: aetherdb_diffusers_kandinsky Size: 14.77 GiB
AetherDB Kokoro 82M Multilingual Baseline	章节朗读、语音预览、发音词典验证。	rocm_pytorch, cuda, cpu	Chapter narration, voice preview, and pronunciation checks. Adapter: aetherdb_torch_kokoro Size: 0.31 GiB

算法流程

Algorithmic Flow

- 实体抽取：导入文本先生成预览，作者确认后写入实体、关系和事件表。

Entity extraction: imported text creates a preview first; confirmed results are written to entity, relation, and event tables.

- 混合检索：关键词检索用于精确命中，向量检索用于语义召回，RAG 组装上下文供 AI 辅助分析。

Hybrid retrieval: keyword search handles exact matches, vector search handles semantic recall, and RAG assembles context for AI-assisted analysis.

- 一致性检查：规则、时间线、关系和设定描述共同参与冲突检测。

Consistency checking: rules, timeline, relations, and lore descriptions jointly contribute to conflict detection.

- 模型调度：任务先通过安装状态、校验记录、资源需求、后端兼容性和租约策略，再进入 Worker。

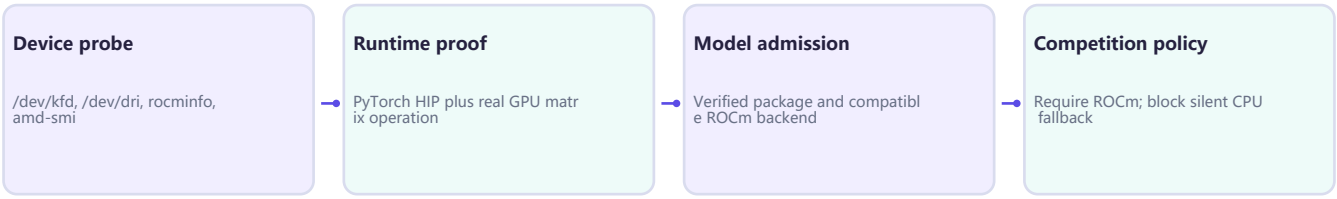
Model scheduling: tasks pass installation state, verification record, resource needs, backend compatibility, and lease policy before entering a worker.

5. AMD Radeon GPU / ROCm 适配说明

5. AMD Radeon GPU / ROCm Adaptation

项目的 ROCm 适配目标是“可证明、可阻断、可复现”。比赛模式下，系统不会把 CPU fallback 伪装成 GPU 成功，也不会允许运行未校验模型。

The ROCm adaptation goal is provable, blockable, and reproducible. In competition mode, the system does not present CPU fallback as GPU success and does not run unverified models.



验证策略

Verification Strategy

- 平台层验证 `/dev/kfd`、`/dev/dri/renderD*`、`rocminfo` 和 `amd-smi`，确认 Linux ROCm 设备可见。

The platform layer verifies `/dev/kfd`, `/dev/dri/renderD*`, `rocminfo`, and `amd-smi` to confirm Linux ROCm device visibility.

- 运行层验证 PyTorch HIP 可用，并执行真实 GPU 矩阵运算，而不是只读取版本号。

The runtime layer verifies PyTorch HIP and performs a real GPU matrix operation rather than only reading version strings.

- 策略层在 competition mode 中要求 `REQUIRE_ROCM=true`、`ALLOW_SILENT_FALLBACK=false`、`ALLOW_UNVERIFIED_MODEL=false`。

The policy layer requires `REQUIRE_ROCM=true`, `ALLOW_SILENT_FALLBACK=false`, and `ALLOW_UNVERIFIED_MODEL=false` in competition mode.

- 调度层按模型声明选择 `rocm_migraphx` 或 `rocm_pytorch`；如果没有兼容 AMD 后端，任务被阻断。

The scheduler selects `rocm_migraphx` or `rocm_pytorch` according to model declarations; without a compatible AMD backend, the task is blocked.

模型后端映射

Model Backend Mapping

- BGE-M3：文本嵌入核心模型，支持 `rocm_migraphx`，也保留 CUDA/CPU 作为非比赛环境路径。

BGE-M3: core text embedding model supporting `rocm_migraphx`, with CUDA/CPU retained for non-competition environments.

- BGE-VL：多模态检索模型，通过 `rocm_pytorch`、CUDA 或 CPU 运行。

BGE-VL: multimodal retrieval model running through `rocm_pytorch`, CUDA, or CPU.

- Kokoro TTS：TTS Worker 通过 `rocm_pytorch`、CUDA 或 CPU 运行，作为轻量语音基线。

Kokoro TTS: TTS worker runs through `rocm_pytorch`, CUDA, or CPU as a lightweight speech baseline.

- Kandinsky 2.2：可选图像生成模型，仅声明 ROCm PyTorch/CUDA，面向高显存环境。

Kandinsky 2.2: optional image generation model declaring ROCm PyTorch/CUDA only, intended for high-VRAM environments.

6. 发行、复现与提交边界

6. Release, Reproducibility, and Submission Boundary

项目发行包采用源码、桌面宿主、后端 sidecar、模型 payload 和安装/卸载脚本分层。打包规则排除 Markdown、运行数据库、日志、缓存、node_modules、dist 和私有 API key。

The release package is layered into source state, desktop host, backend sidecar, model payload, and install/uninstall scripts. Packaging rules exclude Markdown, runtime databases, logs, caches, node_modules, dist, and private API keys.

模型进入发行包前需要通过 manifest 清单中的文件大小与 SHA-256 校验。bge-m3、bge-vl-base 和 kokoro-82m 已按该机制准备为自主管理模型；Kandinsky 作为可选大模型，需要完整离线包后再纳入。

Models must pass manifest-defined file size and SHA-256 verification before entering a release package. bge-m3, bge-vl-base, and kokoro-82m are prepared through this managed-model path; Kandinsky remains optional and requires a complete offline package before inclusion.

提交材料建议

Suggested Submission Materials

- 本 PDF：项目背景、用户场景、架构、模型算法和 AMD ROCm 适配说明。

  This PDF: project background, user scenarios, architecture, models/algorithms, and AMD ROCm adaptation.

- 源码仓库：不包含密钥、本地数据库、日志或发行压缩包大文件，保留配置样例和模型 manifest。

  Source repository: no secrets, local databases, logs, or large release archives; keep config examples and model manifests.

- 演示视频或截图：展示创作工作台、AI 设置、模型包验证、ROCm 状态和检索流程。

  Demo video or screenshots: show the writing workspace, AI settings, model package verification, ROCm status, and retrieval workflow.

- 环境快照：可使用项目脚本导出依赖版本、模型哈希和 ROCm 验证证据。

  Environment snapshot: project scripts can export dependency versions, model hashes, and ROCm verification evidence.

不纳入本文档的内容

Excluded From This Document

本文档不记录本地 API key、私有供应商配置、运行数据库内容、日志、个人路径、测试人员信息或非提交状态下的开发环境异常。

This document does not record local API keys, private provider configs, runtime database contents, logs, personal paths, tester information, or development-environment issues outside the submission state.

7. 结论

7. Conclusion

AetherDB V1.31.2 将职业作者的世界观管理问题转化为一个安全、可验证、可本地运行的工程系统。它用结构化数据库解决设定沉淀，用向量和全文检索解决召回，用可审查 AI 流程解决辅助创作，用 ROCm fail-closed 策略解决比赛环境中的 GPU 可信执行问题。

AetherDB V1.31.2 turns professional worldbuilding management into a secure, verifiable, locally runnable engineering system. It uses a structured database for lore persistence, vector and full-text retrieval for recall, reviewable AI workflows for author assistance, and a ROCm fail-closed strategy for trustworthy GPU execution in the competition environment.

项目当前最适合展示的价值，是面向真实作者工作流的完整闭环：写作、导入、抽取、检索、检查、模型调度、备份和发行。

The strongest current value is a complete loop for real author workflows: writing, import, extraction, retrieval, checking, model scheduling, backup, and release.

评审关注点	AetherDB 回答	Review Value
用户价值	减少世界观维护成本，降低长篇创作中的设定冲突。	Lower lore-maintenance cost and fewer long-form continuity conflicts.
技术完整性	桌面 UI、FastAPI、SQLite、模型包、Worker 和 ROCm 策略形成闭环。	Desktop UI, FastAPI, SQLite, model packages, workers, and ROCm policy form an integrated system.
AMD 相关性	ROCm 不是展示标签，而是被平台探测、运行证明和调度策略共同验证。	ROCm is not a display label; it is verified by platform probes, execution proof, and scheduling policy.
安全性	认证、能力权限、密钥排除和发行包扫描降低误用风险。	Authentication, capabilities, secret exclusion, and release scanning reduce misuse risk.